Formal Methods and Specification
Assignment Sheet 5

Exercise 1

(a) Write down a first-order predicate logical formula that represents the following I/O-specification in the way discussed in the lecture on functions and procedures:

procedure $p(a, r, x)$
Input: $a \in \mathcal{N}, r \in \mathcal{N}, x \in \mathcal{N}$ s.t. $r \geq 0$
Output: a^* s.t. $a^* = a + rx$

(b) Add the formula from the previous item as an assumption to the following program, and convert to program into SSA.

assume $x \geq 8 \wedge k \geq 5$
$p(x, 2, k)$
@ $x \geq 15$

Note that the procedure call may result in a new value for the variable x. Take this into account in the way explained in the lecture. All program variables ranger over the integers.

(c) Write down the verification condition that proves correctness of the program in SSA from the previous item.

(d) Check, whether the verification condition from the previous item holds. You do not have to prove anything, you just should be able to explain why it holds/does not hold.

(e) Prove the verification condition. Here, you may freely use whatever knowledge you have about arrays and integers (i.e., not only the axioms of the corresponding logical theories).

(0.5+0.5+0.5+0.5+1 points)

Exercise 2

Consider the following program:

assume $i \geq 7$
assume $k \geq 0$
while $i \neq 0$ **do**
    @
    if $k \bmod 10 = 0$ **then**
        $i \leftarrow i - 1$; $k \leftarrow 6$
    else
        $k \leftarrow k + 1$

Here, all variables range over the integers and, as usual in this course, the scope of control statements is given by indentation.

(a) Write down a table showing the values of the program variables at the position of the missing loop invariant at all loop iterations for some non-trivial input.

(b) Provide a loop variant wrt. an arbitrary well-founded relation that shows the termination of the program. Explain informally why your result is a variant (no assertions or formal proofs are necessary).

(c) Add assignments and assertions to the program that check that your answer is really a variant.

I explained several similar examples during the lecture on program termination.

(0.5+1+1 points)

Exercise 3

$i \leftarrow k$
while $i \leq n$ **do**
    $i \leftarrow 2i$
@ $i \geq k$

(a) Write down the program that is the result of unrolling the while loop twice. The resulting program will not contain any loop any more, but will contain **if** $-$ **then** statements.

(b) Translate the resulting program into SSA *without* decomposing it into basic paths, keeping the **if** $-$ **then** statements (see the lecture on BMC, the lecture on programs without control structure is *not* enough).

(c) Convert the program in SSA to a logical formula

(1+1+1 points)

Exercise 4

Consider the following program:

$found \leftarrow \bot$
for $i \leftarrow 0$ **to** $len(a) - len(s)$ **do**
  @
  $p \leftarrow \top$
  for $j \leftarrow 0$ **to** $len(s) - 1$ **do**
    @
    if $a[i + j] \neq s[j]$ **then**
      $p \leftarrow \bot$
    @
  if p **then** $found \leftarrow \top$
  @
@ $found \Leftrightarrow \exists r \,.\, substr(a, r, s)$

Here, all variables are integers, except for the variables a and s which are strings. These are defined as pairs consisting of an array of characters and the length of the string with the following operations:

- $\forall\ (A, l) \in \mathcal{S} \,.\, len((A, l)) := l$
- $\forall\ (A, l) \in \mathcal{S} \,.\, (A, l)[i] := A[i]$
- $\forall\ S_1, S_2 \in \mathcal{S} \,.\, S_1 +\!\!+\, S_2 := T$ s.t.
  - $len(T) = len(S_1) + len(S_2)$, and
  - $\forall i \in \{0, \ldots, len(T) - 1\} \,.\, T[i] = \begin{cases} S_1[i], & \text{if } i < len(S_1), \\ S_2[i - len(S_1)], & \text{otherwise.} \end{cases}$
- $\forall S \in \mathcal{S}, p \in \mathcal{N}, S' \in \mathcal{S} \,.\, substr(S, p, S') :\Leftrightarrow$
  $0 \leq p < len(S)$,
  $p + len(S') \leq len(S)$,
  $\forall i \in \{0, \ldots, len(S') - 1\} \,.\, S'[i] = S[p + i]$

(a) Add logical formulas as assertions to the program lines starting with @, trying to ensure that all verification conditions hold. It is *not* important that your answer is correct. But you *must* be able to

- either explain why you think your solution satisfies all verification conditions, or to
- show several possibilities that you tried, explain why those possibilities are not a correct answer, and why you were not able to come up with a correct answer.

(b) Count the number of verification conditions necessary to check correctness of the program. Here, it will not be important to come up with the correct number, but to be able to explain why you think that a certain number is the correct answer.

(c) Write down all verification conditions and check informally, whether they are correct.

(1.5+0.5+1.5 points)